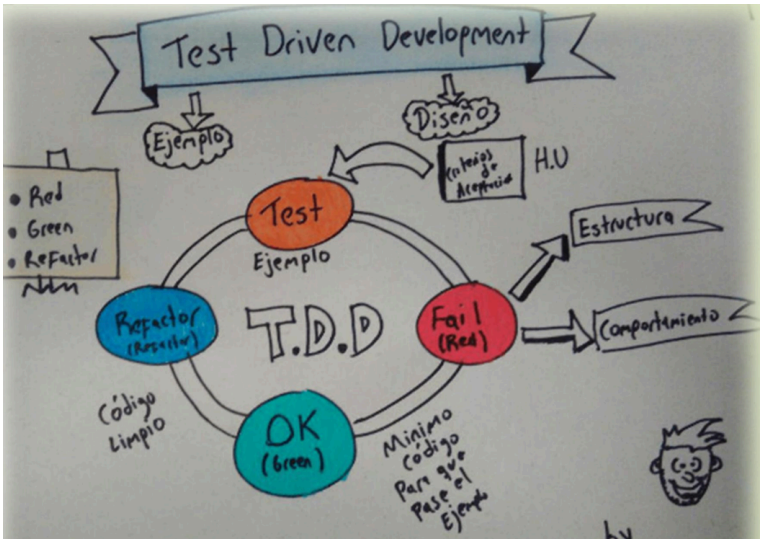
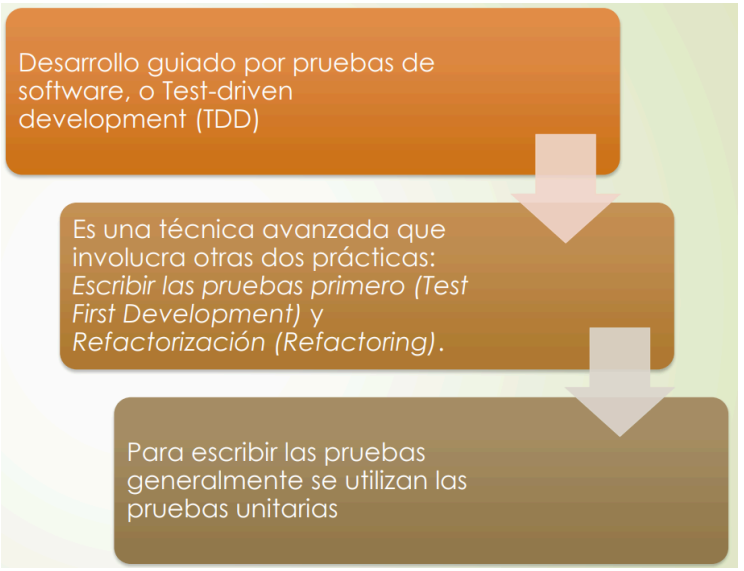
TDD





TDD - TEST DRIVEN DEVELOPMENT - DESARROLLO GUIADO POR PRUEBAS
Es una técnica de desarrollo de software en la que se escriben primero las pruebas y, a partir de ellas, se desarrolla el código de producción necesario para satisfacerlas. No debe confundirse TDD con todo el proceso de testing.

La idea central es cambiar el orden habitual de trabajo: en lugar de programar una funcionalidad y después comprobar si funciona, primero se define mediante una prueba qué comportamiento se espera del software y luego se implementa ese comportamiento.

Técnica que combina dos prácticas:

- **Test First Development:** escribir las pruebas antes que el código de producción.
- **Refactoring:** mejorar la estructura interna del código sin cambiar su comportamiento observable.

Generalmente, las pruebas utilizadas son **pruebas unitarias automatizadas,** que verifican componentes pequeños del software de manera aislada. (pieza de código más pequeña e independiente, como una función, un método o una clase).

Ejemplo: en TDD primero escribimos una prueba que establece ese comportamiento esperado:

```
def test_sumar_dos_numeros():
    assert sumar(2, 3) == 5
```

Todavía no implementamos la función `sumar`. La prueba expresa qué esperamos que haga.
Luego desarrollamos el código necesario para que la prueba pase:

```
def sumar(a, b):
    return a + b
```

Cita de B. Beizer:

“El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores…El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo”

Es decir, pensar cómo vas a probar algo te obliga a entender bien qué tiene que hacer. Eso detecta ambigüedades y errores *antes* de escribir código, no después.

DESARROLLO TRADICIONAL VS TDD

TDD es una técnica que se utiliza en el **desarrollo ágil,** pero no es sinónimo de toda la metodología ágil ni reemplaza todas las actividades de testing.



Enfoque tradicional → Las pruebas unitarias se diseñan después de escribir el código. Diseño, Desarrollo y Pruebo.

Enfoque TDD → Las pruebas se escriben antes del código de producción y guían su desarrollo. Diseño, Pruebo y Desarrollo.

Diferencias:

Aspecto	Desarrollo tradicional (según la PPT)	TDD
Orden de trabajo	Diseñar → desarrollar → probar	Diseñar → probar → desarrollar
Momento de diseñar las pruebas unitarias	Después del código	Antes del código
Aseguramiento y control de calidad	Más reactivo	Más proactivo
Función de las pruebas	Comprobar el código desarrollado	Guiar el desarrollo y comprobar el comportamiento esperado

La diferencia específica es cuándo se escriben las pruebas respecto del código de producción.

¿Qué significa que el control de calidad sea reactivo o proactivo?
- Reactivo: primero se desarrolla el código y después se buscan defectos mediante pruebas. La prueba reacciona ante algo que ya fue construido.
- Proactivo: se anticipan los comportamientos esperados y los posibles problemas antes de implementar la funcionalidad. Las pruebas ayudan a orientar el desarrollo desde el principio. **Previene errores desde el diseño.**

TDD - ATDD

TDD: Se escriben primero **pruebas unitarias** automatizadas que definen el comportamiento esperado de un componente. Está centrado principalmente en el diseño y la verificación del comportamiento a **nivel de código.**

ATDD: Antes de desarrollar una funcionalidad, se definen los **criterios y las pruebas de aceptación** que permitirán determinar si satisface las **necesidades del usuario o los requisitos del negocio.**

Idea para entender: el test de aceptación responde *"¿hace lo que el cliente pidió?"*. Los unitarios responden *"¿funciona bien cada pieza?"*. **Para hacer pasar un test de aceptación, hacés muchos ciclos TDD chiquitos.**

BENEFICIOS TDD

Código más robusto: no se rompe fácilmente, cómo pensás los escenarios de falla *antes* de programar el código nace preparado para esos casos.

Código más predecible y limpio: es predecible porque hace lo que los test dicen, ni más ni menos. Y permite saber cuándo el código está terminado.
En tradicional es difícil responder "¿ya está?". En TDD tenés una respuesta objetiva: cuando todos los tests pasan, la funcionalidad está lista. Los tests funcionan como la definición de "terminado".

Código más tolerable al cambio: al tener escritas las pruebas es más fácil de entender. Es decir, las pruebas ayudan a detectar si una modificación rompe algo que funcionaba. Si cambias algo y rompes otra parte, un test se pone en rojo.

Código más seguro: confianza en que el código hace lo que debe. Como cada comportamiento está verificado por un test, tenés evidencia de que funciona.

Aumenta la posibilidad de **reducir la duplicación de código:** si el test pasa el código ya existe. Cuando escribes un test nuevo y ya pasa sin escribir nada, significa que esa funcionalidad ya existía en alguna parte. Te avisa antes de que programes lo mismo dos veces. Además, el paso Refactor del ciclo tiene como objetivo explícito quitar duplicaciones.

Código más barato de mantener: es consecuencia de los beneficios anteriores. El mantenimiento suele ser la etapa más cara de un software. Un error encontrado tarde cuesta mucho más que uno encontrado al escribir el código.

Con la práctica **acelera el desarrollo:** pierdes menos tiempo depurando (buscando errores a mano), tenés menos errores que corregir después.

Facilita el despliegue continuo: es posible porque los tests automatizados confirman en minutos que nada se rompió. Sin tests, desplegar seguido es arriesgado.

Asegura cobertura de prueba: conlleva a código de calidad. En tradicional, los tests se escriben al final y siempre se "quedan sin tiempo", así que quedan partes sin probar. En

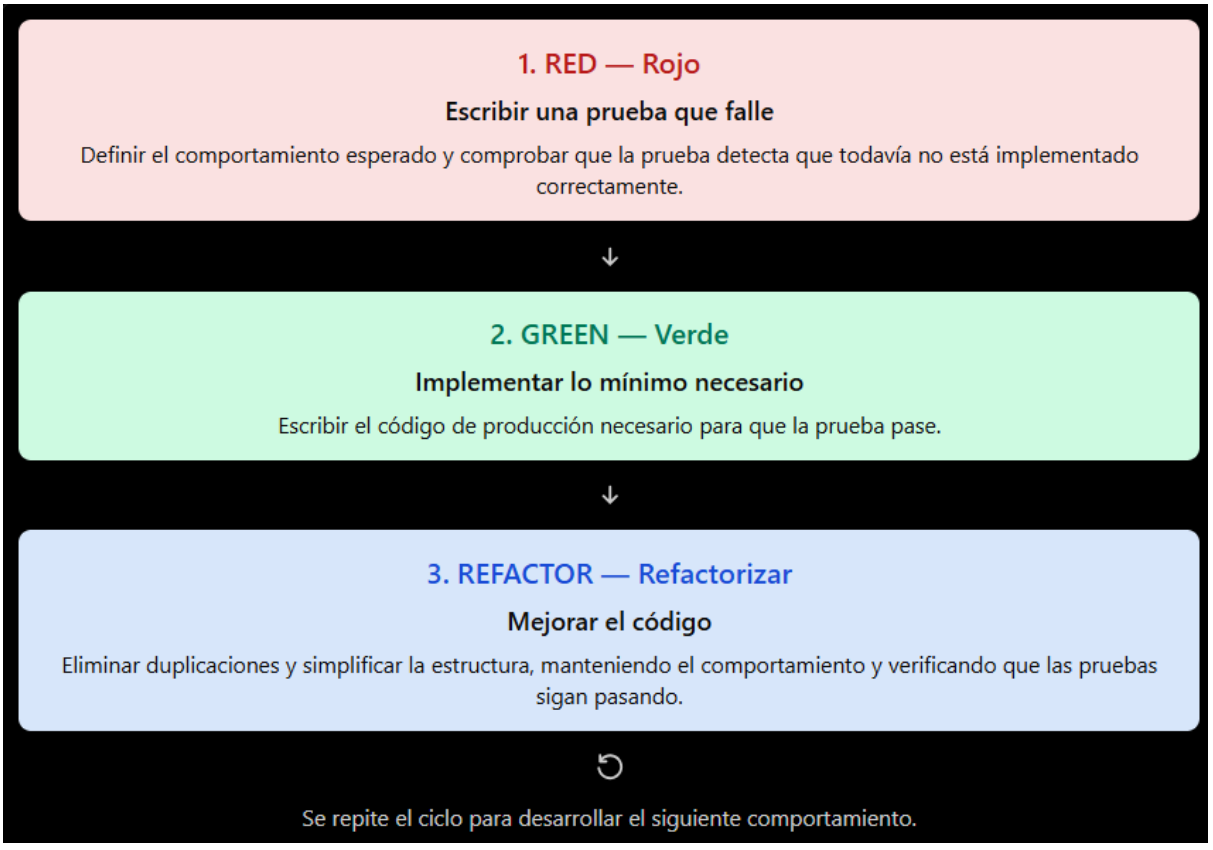
TDD no existe código sin test, porque cada línea de producción se escribió para hacer pasar un test (1ª y 3ª ley). La cobertura sale casi automática.

Obliga a pensar en cómo usar el componente, por lo tanto, el **diseño de las API:** con diseñar la API se refiere a la interfaz: la forma en que otros usan tu componente. Si es incómoda de usar en el test, lo notás enseguida y la mejoras *antes* de implementarla.

Promueve el desarrollo de componentes con **alta cohesión y bajo acoplamiento:** son dos conceptos de diseño importantes. Alta cohesión: cada componente hace una sola cosa y todo lo que tiene está relacionado con eso. Bajo acoplamiento: los componentes dependen poco unos de otros, así que podés cambiar uno sin romper los demás.

CICLO Red-Green-Refactor

El Corazón de TDD. El desarrollo guiado por pruebas funciona mediante un ciclo corto que se repite durante la implementación de cada funcionalidad.



Refactoring

Se trata de una transformación del software que mejora su estructura interna y preserva su estructura externa o comportamiento observable.

En otras palabras, **mejorar cómo está escrito el código sin cambiar lo que hace desde el punto de vista de quien lo utiliza.**

El objetivo de la refactorización es que las pruebas continúen pasando. Es una forma **disciplinada** de introducir cambios **reduciendo el riesgo de meter defectos.**

La refactorización permite:

- Eliminar duplicación y complejidad innecesaria.
- Mejorar la calidad, legibilidad y comprensión del código.
- Facilitar futuros cambios.
- Evitar el deterioro del diseño.
- Reducir el tiempo de depuración y ayudar a encontrar errores.

TDD no busca únicamente conseguir que el código funcione. También incorpora un momento explícito para mejorar su diseño.

¿Cómo hacer Refactoring?



"Asegurarse de que las pruebas pasen" → **por eso va junto con TDD**: sin tests no sabés si rompiste algo.

LAS 3 LEYES DE TDD

Reglas que **limitan cuánto código** y **cuántas pruebas** escribimos en cada ciclo.

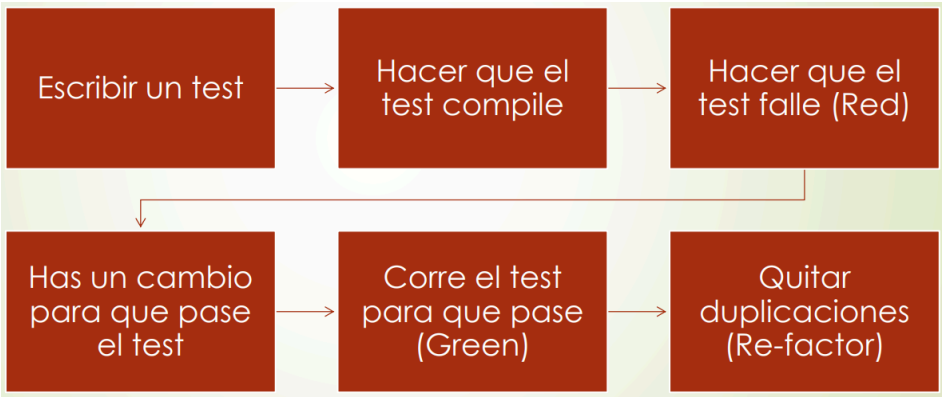
1. No escribir código de producción sin haber escrito antes un caso de prueba que falle. Primero definimos y comprobamos el comportamiento esperado. Después lo implementamos.

2. No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente. Es decir: escribís el test de a poquito, solo lo suficiente para que falle (incluso un error de compilación cuenta como fallo). No se trata de "cuántos tests en total", sino de avanzar en pasos chicos: un test, un fallo, y recién ahí codificás.

3. No vas a escribir más código en Producción que el necesario para que el test pase. Se evita implementar funcionalidades o agregar complejidad que todavía no está justificada por las pruebas. Si todas las pruebas pasan, tenés el código que necesitás, ni más ni menos.

   En el práctico: por funcionalidad, debemos tener 1 test que pasa y 1 que falla. Por ende vamos a terminar con varios test, pero la ley en sí habla de **ir de a un test por vez**.

PASOS DEL DESARROLLO CONDUCIDO POR TESTING (TDD)



PATRONES

One Step Test: escribir una prueba para una pieza mínima de funcionalidad.
Starter Test: comenzar con una prueba que ayude a explorar el comportamiento deseado o la API.
Learning Test: utilizar pruebas para aprender cómo se comporta una API externa.
Regression Test: agregar una prueba que reproduzca un error encontrado para evitar que reaparezca.
Fake It: implementar temporalmente una solución sencilla o un valor fijo para hacer pasar la prueba.
Triangulación: agregar otra prueba con datos diferentes para obligar a desarrollar una solución más general.
Obvious Implementation: implementar directamente la solución cuando resulta evidente.
Isolated Test: mantener las pruebas independientes entre sí.